



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第四章 数组、串与广义表



- 一维数组与多维数组
- 特殊矩阵
- 稀疏矩阵
- 字符串
- 广义表



一维数组（顺序表）



- 定义

数组是相同类型的数据元素的集合，而一维数组的每个数组元素是一个序对，由下标（index）和值（value）组成。

- 一维数组的示例

0	1	2	3	4	5	6	7	8	9
35	27	49	18	60	54	77	83	41	02

- 在高级语言中的一维数组只能按元素的下标直接存取数组元素的值，而不能一次引用整个数组。



一维数组的定义和初始化



```
#include <iostream.h>
```

```
void main ( ) {
```

```
    int a[3] = { 3, 5, 7 }, *elem, i;
```

//静态数组

```
    for (i = 0; i < 3; i++)
```

```
        cout << a[i] << endl;
```

```
    elem = new int[3];
```

//动态数组

```
    for (i = 0; i < 3; i++)
```

```
        cin >> elem[i];
```

```
    int * temp = elem;
```

```
    for (i = 0; i < 3; i++)
```

```
        { cout << *temp<< endl; temp++; }
```

```
    delete [] elem;
```

```
}
```



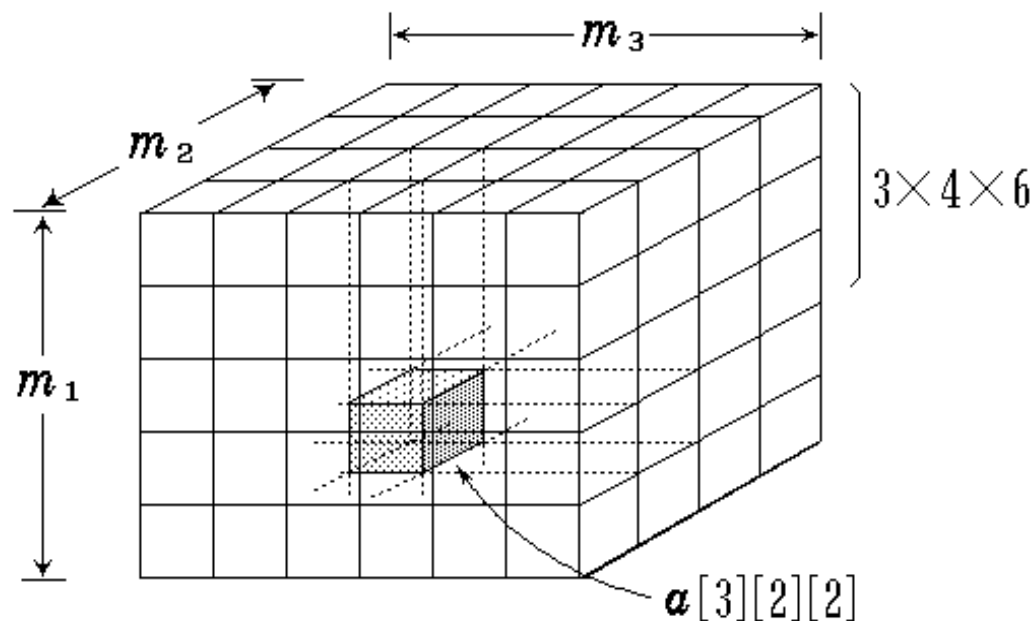
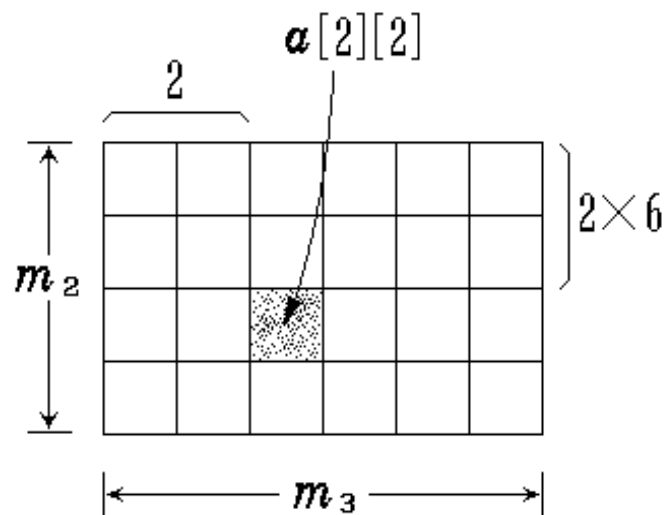
4.1 多维数组



- 多维数组是一维数组的推广。
- 多维数组的特点是每一个数据元素可以有**多个直接前驱**和**多个直接后继**。
- 数组元素的下标一般具有固定的下界和上界，因此，它比其他复杂的非线性结构简单。
 - 例如，二维数组的数组元素有两个直接前驱、两个直接后继，必须有两个下标（行、列）以标识该元素的位置。



$$m_1 = 5 \quad m_2 = 4 \quad m_3 = 6$$



- 行向量 下标 i
- 列向量 下标 j
-

- 页向量 下标 i
- 行向量 下标 j
- 列向量 下标 k

二维数组

三维数组

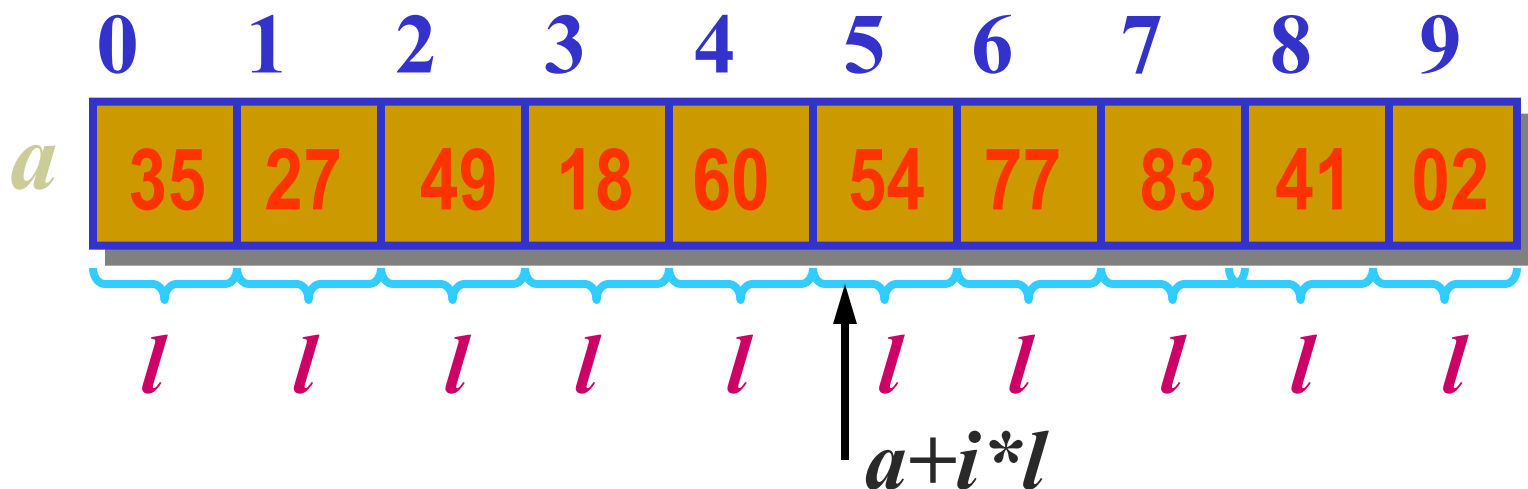


数组的连续存储方式



一维数组

$$\text{LOC}(i) = \begin{cases} a, & i = 0 \\ \text{LOC}(i-1) + l = a + i * l, & i > 0 \end{cases}$$





二维数组



- **一维数组**常被称为向量 (Vector) 。
- **二维数组** $A[m][n]$ 可看成是由 m 个行向量组成的向量，也可看成是由 n 个列向量组成的向量。
- 一个**二维数组**类型可以定义为其分量类型为一维数组类型的一维数组类型：

```
typedef T array2[m][n];    //T为元素类型
```

等价于：

```
typedef T array1[n];       //列向量类型
```

```
typedef array1 array2[m];  //二维数组类型
```




- 同理，一个**三维数组**类型可以定义为其数据元素为二维数组类型的一维数组类型。
- **静态定义**的数组，其维数和维界不再改变，在编译时静态分配存储空间。一旦数组空间用完则不能扩充。
- **动态定义**的数组，其维界不在说明语句中显式定义，而是在程序运行中创建数组对象时通过 `new` 动态分配和初始化，在对象销毁时通过 `delete` 动态释放。
- 用**一维内存**来表示**多维数组**，就必须按某种次序将数组元素排列到一个序列中。



二维数组的动态定义和初始化



```
#include <iostream.h>
```

```
.....
```

```
int **A;
```

```
int row = 3, col = 3;  int i, j;
```

```
A = new int *[row];
```

```
for (i = 0; i < row; i++)
```

```
    A[i] = new int [col];
```

```
for (i = 0; i < row; i++)
```

```
    for (j = 0; j < col; j++)  cin >> A[i][j];    //动态定义
```

```
.....
```



二维数组A[n][m]中数组元素的顺序存放



$$a = \begin{pmatrix} a[0][0] & a[0][1] & \boxed{?} & a[0][m-1] \\ a[1][0] & a[1][1] & \boxed{?} & a[1][m-1] \\ a[2][0] & a[2][1] & \boxed{?} & a[2][m-1] \\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?} \\ a[n-1][0] & a[n-1][1] & \boxed{?} & a[n-1][m-1] \end{pmatrix}$$

Row major order: $LOC(i, j) = a + (i * m + j) * l$

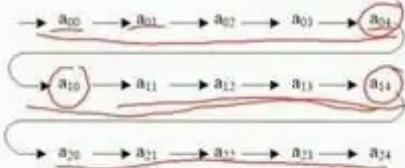
Column major order: $LOC(i, j) = a + (j * n + i) * l$



二维数组的存储：按行连续存放

□ 行优先的存储结构

`float a[3][5];`



□ 设数组a的首地址 $p = \&a[0][0]$ ，则
 $a[i][j]$ 的地址是： $p + (i * \text{col} + j) * 4$

$p \rightarrow$	$a[0][0]$	00242000
	$a[0][1]$	00242004
	$a[0][2]$	00242008
	$a[0][3]$	0024200C
	$a[0][4]$	00242010
	$a[1][0]$	00242014
	$a[1][1]$	00242018
	$a[1][2]$	0024201C
	$a[1][3]$	00242020
	$a[1][4]$	00242024
	$a[2][0]$	00242028
	$a[2][1]$	0024202C
	$a[2][2]$	00242030
	$a[2][3]$	00242034
	$a[2][4]$	00242038

$p + (2 * 5 + 1) * 4$

- **行优先存放：**
设数组开始存放位置 $\text{LOC}(0, 0) = a$ ，每个元素占用 l 个存储单元
- $\text{LOC}(j, k) = a + (j * m + k) * l$



Row-major order
e.g., C

$\begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \end{bmatrix}$

Address	Coordinates	Value
0	[1, 1]	11
1	[1, 2]	12
2	[1, 3]	13
3	[2, 1]	21
4	[2, 2]	22
5	[2, 3]	23

Column-major order
e.g., Fortran

Address	Coordinates	Value
0	[1, 1]	11
1	[2, 1]	21
2	[1, 2]	12
3	[2, 2]	22
4	[1, 3]	13
5	[2, 3]	23

列优先存放:

设数组开始存放位置 $LOC(0, 0) = a$, 每个元素占用 l 个存储单元

$$LOC(j, k) = a + (k * n + j) * l$$



三维数组



- 各维元素个数为 m_1, m_2, m_3
- 下标为 i_1, i_2, i_3 的数组元素的存储地址：
(按页 / 行 / 列存放)

$$\text{LOC} (i_1, i_2, i_3) = a + \underbrace{(i_1 * m_2 * m_3)}_{\text{前 } i_1 \text{ 页 总元素 个数}} + \underbrace{i_2 * m_3}_{\text{第 } i_1 \text{ 页 前 } i_2 \text{ 行 总元素 个数}} + \underbrace{i_3}_{\text{第 } i_2 \text{ 行 前 } i_3 \text{ 列 元素 个数}} * l$$

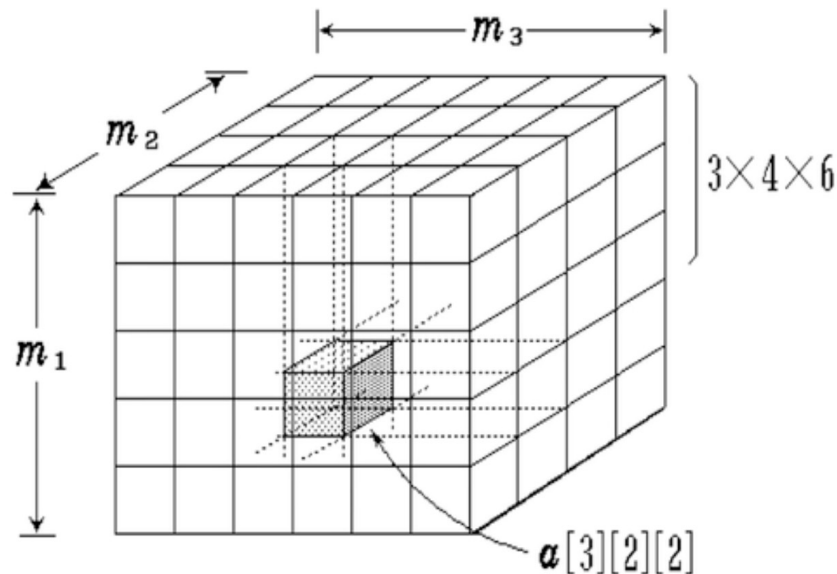


? dimension values are m_1, m_2, m_3

? the address of the element associated with

index i, j, k

$$LOC(i, j, k) = a + (i * m_2 * m_3 + j * m_3 + k) * I$$





n 维数组



- 各维元素个数为 $m_1, m_2, m_3, \dots, m_n$
- 下标为 $i_1, i_2, i_3, \dots, i_n$ 的数组元素的存储地址:

$$\text{LOC} (i_1, i_2, \dots, i_n) = a +$$

$$(\textcolor{yellow}{i_1} * m_2 * m_3 * \dots * m_n + \textcolor{yellow}{i_2} * m_3 * m_4 * \dots * m_n + \\ + \dots + \textcolor{yellow}{i_{n-1}} * m_n + \textcolor{yellow}{i_n}) * l$$

$$= c + \left(\sum_{l=3}^{p-3} *k_l * \prod_{m=l+3}^p o_m + k_p \right), n$$

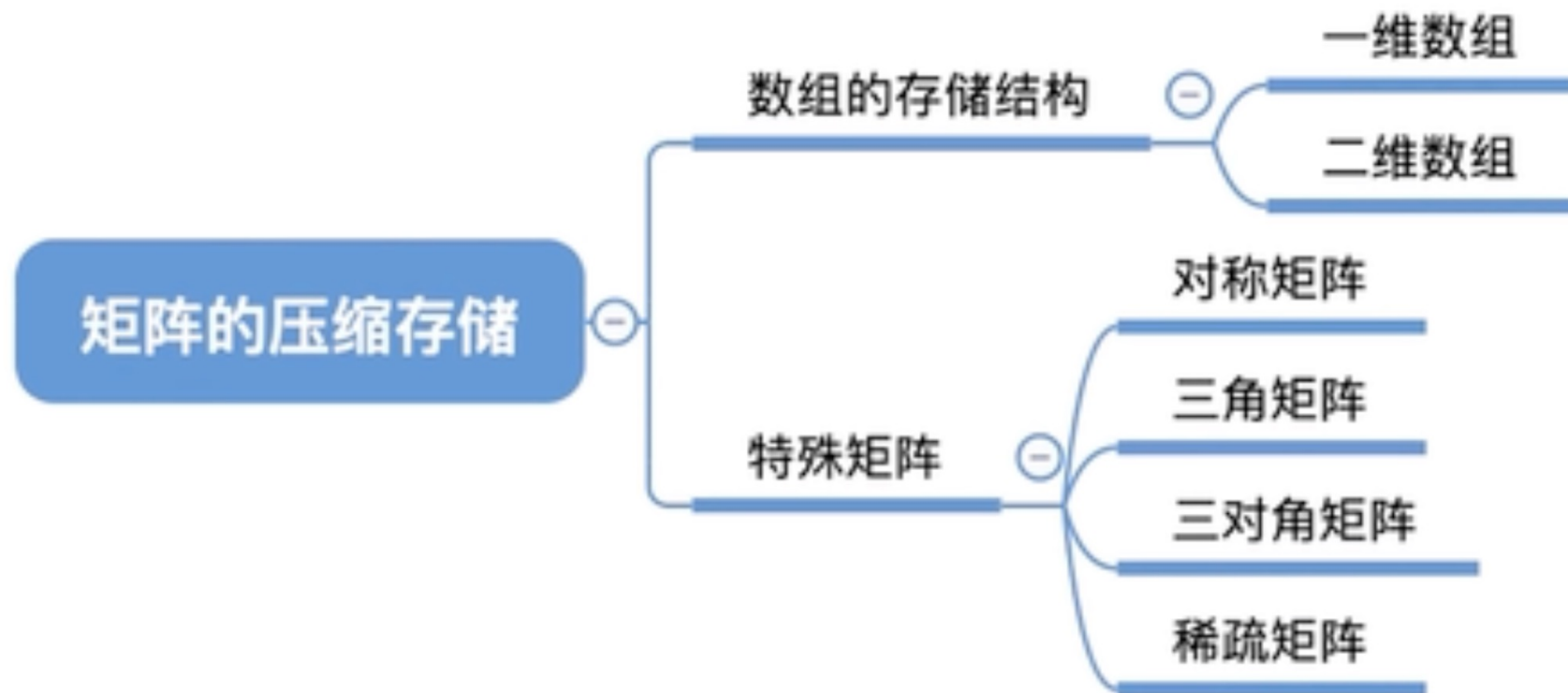


❑ dimension values are $m_1, m_2, m_3, \dots, m_n$

❑ the address of the element associated with index $i_1, i_2, i_3, \dots, i_n$

$$\begin{aligned} LOC(i_1, i_2, \dots, i_n) = & a + \\ & (i_1 * m_2 * m_3 * \dots * m_n + i_2 * m_3 * m_4 * \dots * m_n + \dots \\ & + i_{n-1} * m_n + i_n) * I \end{aligned}$$

$$= a + \left(\sum_{j=1}^{n-1} i_j * \prod_{k=j+1}^n m_k + i_n \right) * I$$





4.2 特殊矩阵



- **特殊矩阵**是指非零元素或零元素的分布有一定规律的矩阵。
- 特殊矩阵的压缩存储主要是针对阶数很高的特殊矩阵。为节省存储空间，对可以不存储的元素，如**零**元素或**对称**元素，不再存储。
 - ◆ 对称矩阵
 - ◆ 三对角矩阵



4.2.1 对称矩阵的压缩存储



- 设有一个 $n \times n$ 的对称矩阵 A 。

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

- 对称矩阵中的元素关于主对角线对称,
 $a_{ij} = a_{ji}, \quad 0 \leq i, j \leq n-1$



- 为节约存储，只存对角线及对角线以上的元素，或者只存对角线及对角线以下的元素。前者称为**上三角矩阵**，后者称为**下三角矩阵**。



下
三
角
矩
阵



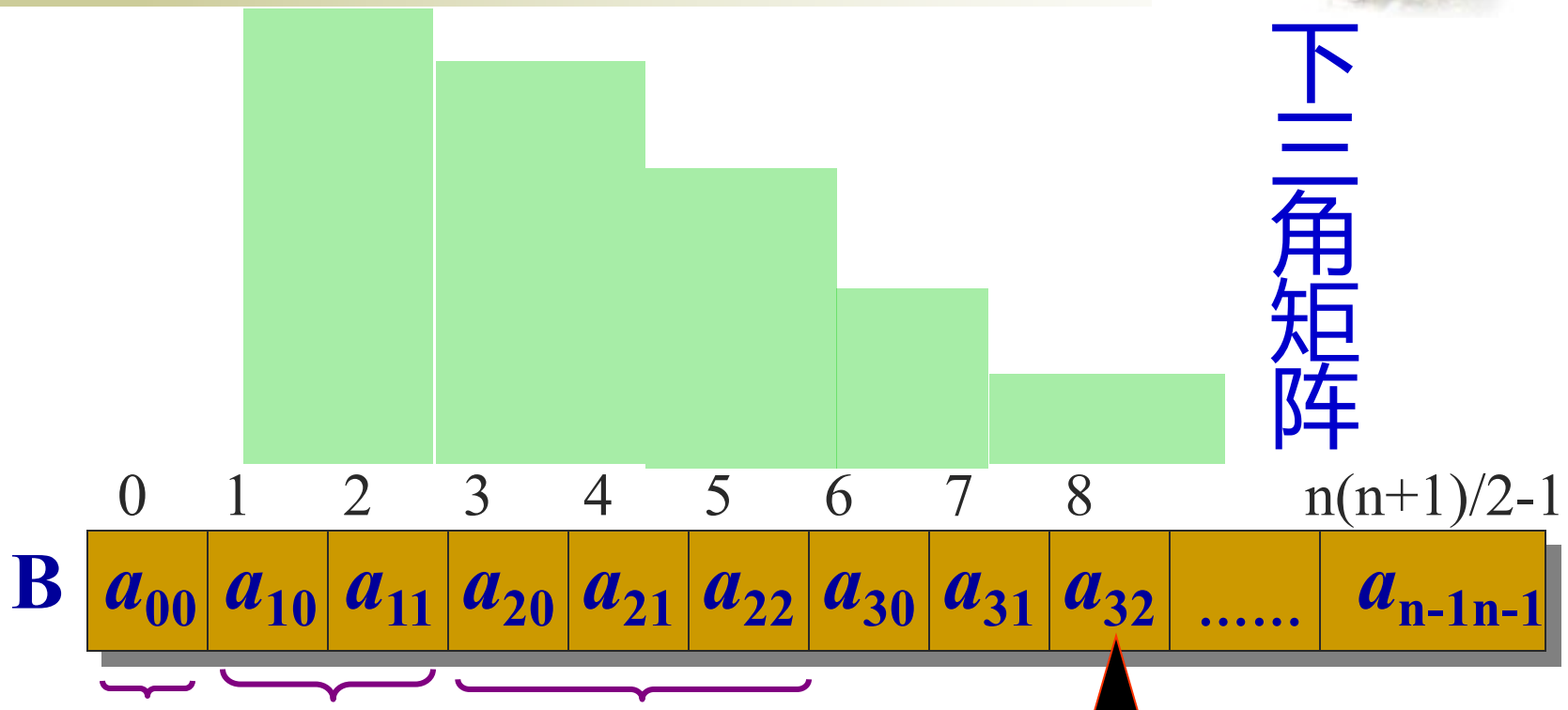
上三角矩阵



- 把它们按行存放于一个一维数组 B 中，称之为对称矩阵 A 的压缩存储方式。
- 数组 B 共有 $n + (n - 1) + \cdots + 1 = n*(n+1) / 2$ 个元素。



下三角矩阵



若 $i \geq j$, 数组元素 $A[i][j]$ 在数组 B 中的存放位置为 $1 + 2 + \cdots + i + j = (i + 1) * i / 2 + j$

前 i 行元素总数 第 i 行第 j 个元素前元素个数



- 若 $i < j$, 数组元素 $A[i][j]$ 在矩阵的上三角部分, 在数组 B 中没有存放, 可以找它的对称元素 $A[j][i]$: $= j * (j + 1) / 2 + i$
- 若已知某矩阵元素位于数组 B 的第 k 个位置 ($k \geq 0$), 可寻找满足
$$i(i + 1) / 2 \leq k < (i + 1) * (i + 2) / 2$$
的 i , 此即为该元素的行号。
$$j = k - i * (i + 1) / 2$$
此即为该元素的列号。
- 例, 当 $k = 8$, $3 * 4 / 2 = 6 \leq k < 4 * 5 / 2 = 10$, 取 $i = 3$ 。则 $j = 8 - 3 * 4 / 2 = 2$ 。



$n = 4$

上三角矩阵

	0	1	2	3	4	5	6	7	8	9
B	a_{00}	a_{01}	a_{02}	a_{03}	a_{11}	a_{12}	a_{13}	a_{22}	a_{23}	a_{33}

若 $i \leq j$, 数组元素 $A[i][j]$ 在数组 B 中的存放位置为

$$n + (n-1) + (n-2) + \cdots + (n-i+1) + j-i$$

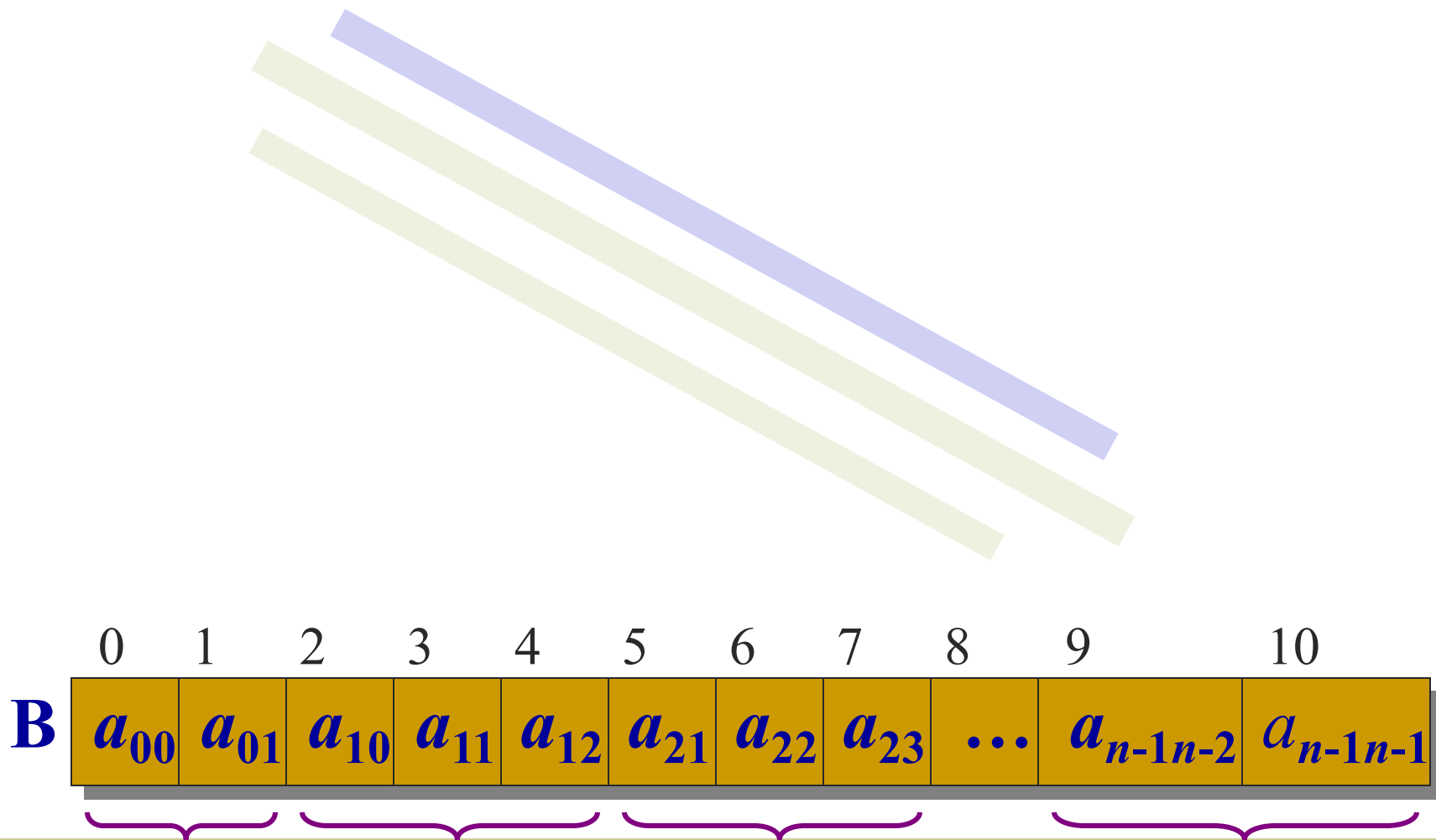
前 i 行元素总数 第 i 行第 j 个元素前元素个数



- 若 $i \leq j$, 数组元素 $A[i][j]$ 在数组 B 中的存放位置为
$$\begin{aligned} & n + (n-1) + (n-2) + \cdots + (n-i+1) + j-i = \\ & = (2*n-i+1) * i / 2 + j-i = \\ & = (2*n-i-1) * i / 2 + j \end{aligned}$$
- 若 $i > j$, 数组元素 $A[i][j]$ 在矩阵的下三角部分, 在数组 B 中没有存放。因此, 找它的对称元素 $A[j][i]$ 。 $A[j][i]$ 在数组 B 的第 $(2*n-j-1) * j / 2 + i$ 的位置中找到。



4.2.2 三对角矩阵的压缩存储





- **三对角矩阵**中除主对角线及在主对角线上下最临近的两条对角线上的元素外，所有其它元素均为0。总共有 $3n-2$ 个非零元素。
- 将三对角矩阵A中三条对角线上的元素按行存放在一维数组 B 中，且 a_{00} 存放于B[0]。
- 在三条对角线上的元素 a_{ij} 满足
$$0 \leq i \leq n-1, \quad i-1 \leq j \leq i+1$$
- 在一维数组 B 中 $A[i][j]$ 在第 i 行，它前面有 $3*i-1$ 个非零元素, 在本行中第 j 列前面有 $j-i+1$ 个，所以元素 $A[i][j]$ 在 B 中位置为 $k = 2*i + j$ 。



- 若已知三对角矩阵中某元素 $A[i][j]$ 在数组 $B[]$ 存放于第 k 个位置, 则有

$$i = \lfloor (k + 1) / 3 \rfloor$$

$$j = k - 2 * i$$

- 例如, 当 $k = 8$ 时,

$$i = \lfloor (8+1) / 3 \rfloor = 3, j = 8 - 2*3 = 2$$

当 $k = 10$ 时,

$$i = \lfloor (10+1) / 3 \rfloor = 3, j = 10 - 2*3 = 4$$



设 A 是一个 $n \times n$ 的下三角矩阵（即当 $i < j$ 时， $A[i, j] = 0$ ）。如果将矩阵的所有非零元素（包括主对角线上的元素）按行优先的顺序压缩存储到一维数组 $B[0..m]$ 中，那么非零元素 $A[i, j]$ （其中 $0 \leq j \leq i \leq n - 1$ ）在数组 B 中对应的下标 k 为 _____。

（请用含 i, j 的表达式表示）

设 A 是一个 $n \times n$ 的上三角矩阵（即当 $i > j$ 时， $A[i, j] = 0$ ）。如果将矩阵的所有非零元素（包括主对角线上的元素）按行优先的顺序压缩存储到一维数组 $B[0..m]$ 中，那么非零元素 $A[i, j]$ （其中 $0 \leq i \leq j \leq n - 1$ ）在数组 B 中对应的下标 k 为 _____。

（请用含 i, j, n 的表达式表示）

已知一个 $n \times n$ 的三对角矩阵 A ，其非零元素只分布在主对角线、第一次对角线和第一次下对角线上。如果将该矩阵的所有非零元素按行优先方式压缩存储到一维数组 $B[0..k]$ 中，那么矩阵元素 $A[i, j]$ （其中 $0 \leq i, j \leq n - 1$ ）在数组 B 中的位置索引 k 为 _____。

（请用含 i, j 的表达式表示）



4.3 稀疏矩阵 (Sparse Matrix)



s p a r s e

0	7	0	0	0	0	6
0	7	6	3	0	4	0
0	4	3	0	0	0	0
4	2	0	0	0	0	0
0	0	0	0	3	2	4

DENSE

0	7	0	0	0	0	6
0	7	6	3	0	4	0
0	4	3	0	0	0	0
4	2	0	0	0	0	0
0	0	0	0	3	2	4

CSDN@独影月下酌酒

- 设矩阵 A 中有 s 个非零元素，若 s 远远小于矩阵元素的总数（即 $s \ll m \times n$ ），则称 A 为**稀疏矩阵**。



- 设矩阵 A 中有 s 个非零元素。令 $e = s/(m \times n)$, 称 e 为矩阵的**稀疏因子**。
- 有人认为 $e \leq 0.05$ 时称之为**稀疏矩阵**。
- 在存储稀疏矩阵时, 为节省存储空间, 应只存储非零元素。但由于非零元素的分布一般没有规律, 故在存储非零元素时, 必须记下它所在的行和列的位置 (i, j) 。
- 每一个**三元组** (i, j, a_{ij}) 唯一确定了矩阵 A 的一个非零元素。因此, 稀疏矩阵可由表示非零元素的一系列**三元组**及其**行列数**唯一确定。



4.3.1 稀疏矩阵的定义



```
const int drows = 6, dcols = 7, dterms = 9;
template<class E>
struct Triple {                                //三元组
    int row, col;                             //非零元素行号/列号
    E value;                                  //非零元素的值
    void operator = (Triple<E>& R)           //赋值
    { row = R.row; col = R.col; value = R.value; }
};
```



```
template <class E>
class SparseMatrix {
public:
    SparseMatrix (int Rw = drows, int Cl = dcols,
                  int Tm = dterms);           //构造函数
    void Transpose(SparseMatrix<E>& b);       //转置
    void Add (SparseMatrix<E>& a,
              SparseMatrix<E>& b);           //a = a+b
    void Multiply (SparseMatrix<E>& a,
                  SparseMatrix<E>& b);       //a = a*b
private:
    int Rows, Cols, Terms;                   //行 / 列 / 非零元素数
    Triple<E> *smArray;                      //三元组表
};
```



稀疏矩阵的构造函数



```
template <class E>
SparseMatrix<E>::SparseMatrix (int Rw, int Cl, int
    Tm) {
    Rows = Rw;  Cols = Cl;  Terms = Tm;
    smArray = new Triple[Terms];      //三元组表
    if (smArray == NULL) {
        cerr << “存储分配失败! ” << endl; exit(1);
    }
};
```



4.3.2 稀疏矩阵的转置



- 一个 $m \times n$ 的矩阵 A ，它的转置矩阵 B 是一个 $n \times m$ 的矩阵，且 $A[i][j] = B[j][i]$ 。即
 - ◆ 矩阵 A 的行成为矩阵 B 的列；
 - ◆ 矩阵 A 的列成为矩阵 B 的行。
- 在稀疏矩阵的三元组表中，非零矩阵元素按行存放。当行号相同时，按列号递增的顺序存放。
- 稀疏矩阵的转置运算要转化为对应三元组表的转置。



稀疏矩阵



$$\begin{pmatrix} 0 & 0 & 0 & 22 & 0 & 0 & 15 \\ 0 & 11 & 0 & 0 & 0 & 17 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 39 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 & 0 \end{pmatrix}$$

	行 (row)	列 (col)	值 (value)
[0]	0	3	22
[1]	0	6	15
[2]	1	1	11
[3]	1	5	17
[4]	2	3	-6
[5]	3	5	39
[6]	4	0	91
[7]	5	2	28



转置矩阵



$$\begin{pmatrix} 0 & 0 & 0 & 22 & 0 & 0 & 15 \\ 0 & 11 & 0 & 0 & 0 & 17 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 39 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 & 0 \end{pmatrix}^T$$

	行 (row)	列 (col)	值 (value)
[0]	0	4	91
[1]	1	1	11
[2]	2	5	28
[3]	3	0	22
[4]	3	2	-6
[5]	5	1	17
[6]	5	3	39
[7]	6	0	15



用三元组表表示的稀疏矩阵及其转置



原矩阵三元组表

	行 (row)	列 (col)	值 (value)
[0]	0	3	22
[1]	0	6	15
[2]	1	1	11
[3]	1	5	17
[4]	2	3	-6
[5]	3	5	39
[6]	4	0	91
[7]	5	2	28

转置矩阵三元组表

	行 (row)	列 (col)	值 (value)
[0]	0	4	91
[1]	1	1	11
[2]	2	5	28
[3]	3	0	22
[4]	3	2	-6
[5]	5	1	17
[6]	5	3	39
[7]	6	0	15



稀疏矩阵转置算法思想



- 设矩阵列数为 **Cols**，对矩阵三元组表扫描 **Cols** 次。
- 第 **k** 次扫描找寻所有列号为 **k** 的项，将其行号变列号、列号变行号，顺次存于转置矩阵三元组表。



稀疏矩阵的转置



```
template <class E>
void SparseMatrix<E> ::
Transpose (SparseMatrix<E>& B) {
//转置this矩阵，转置结果由B返回
    B.Rows = Cols;  B.Cols = Rows;
    B.Terms = Terms;
    //转置矩阵的列数、行数和非零元素个数
    if (Terms > 0) {
        int CurrentB = 0; //转置三元组表存放指针
        int i, k;
```



```
for (k = 0; k < Cols; k++)    //处理所有列号
    for (i = 0; i < Terms; i++)
        if (smArray[i].col == k) {
            B.smArray[CurrentB].row = k;
            B.smArray[CurrentB].col =
                smArray[i].row;
            B.smArray[CurrentB].value=
                smArray[i].value;
            CurrentB++;
        }
    }
};
```



- 设矩阵三元组表总共有 t 项，上述算法的时间代价为 $O(n * t)$ 。当非零元素的个数 t 和 $m * n$ 同数量级时，算法Transpose的时间复杂度为 $O(m * n^2)$ 。
- 若矩阵有 200 行，200 列，10,000 个非零元素，总共有 2,000,000 次处理。



快速转置算法



- 快速转置算法的思想为：
 - 对原矩阵A 扫描一遍（实际为两遍，建辅助数组需扫描一遍），按 A 中每一元素的列号，立即确定在转置矩阵 B 三元组表中的位置，并装入它。
 - 为加速转置速度，建立辅助数组 **rowSize** 和 **rowStart**：
 - rowSize**记录矩阵转置前各列，即转置矩阵各行非零元素个数；
 - rowStart**记录转置矩阵各行非零元素在转置三元组表中开始存放位置。
 - 扫描矩阵三元组表，根据某项列号，确定它转置后的行号，查 **rowStart** 表，按查到的位置直接将该项存入转置三元组表中。



	[0]	[1]	[2]	[3]	[4]	[5]	[6]	语义
rowSize	1	1	1	2	0	2	1	矩阵 A 各列 非零元素个数

rowStart	0	1	2	3	5	5	7	矩阵 B 各行 开始存放位置
----------	---	---	---	---	---	---	---	-------------------

0	0	0	22	0	0	15
0	11	0	0	0	17	0
0	0	0	-6	0	0	0
0	0	0	0	0	39	0
91	0	0	0	0	0	0
0	0	28	0	0	0	0

A三元组	(0)	(1)	(2)	(3)	(4)	(5)	(6)	(7)
行row	0	0	1	1	2	3	4	5
列col	3	6	1	5	3	5	0	2
值value	22	15	11	17	-6	39	91	28



稀疏矩阵的快速转置算法



```
template <class E>
void SparseMatrix<E>::
    FastTranspose (SparseMatrix<E>& B) {
    int *rowSize = new int[Cols];    //列元素数数组
    int *rowStart = new int[Cols];  //转置位置数组
    B.Rows = Cols;  B.Cols = Rows;
    B.Terms = Terms;
    if (Terms > 0) {
        int i, j;
        for (i = 0; i < Cols; i++) rowSize[i] = 0;
```



```
for (i = 0; i < Terms; i++)
    rowSize[smArray[i].col]++;
rowStart[0] = 0;
for (i = 1; i < Cols; i++)
    rowStart[i] = rowStart[i-1]+rowSize[i-1];
for (i = 0; i < Terms; i++) {
    j = rowStart [smArray[i].col];
    B.smArray[j].row = smArray[i].col;
    B.smArray[j].col = smArray[i].row;
    B.smArray[j].value = smArray[i].value;
    rowStart [smArray[i].col]++;
}
}
delete [ ] rowSize;  delete [ ] rowStart;
}
```



带行指针数组的二元组表



- 稀疏矩阵的三元组表可以用带行指针数组的二元组表代替。
- 在行指针数组中元素个数与矩阵行数相等。第 i 个元素的下标 i 代表矩阵的第 i 行，元素的内容即为稀疏矩阵第 i 行的第一个非零元素在二元组表中的存放位置。



- 二元组表中每个二元组只记录非零元素的列号和元素值, 且各二元组按行号递增的顺序排列。

行指针数组

	row
0	0
1	3
2	4
3	6
4	7
5	-1

二元组表 data

	col	value
0	0	12
1	2	11
2	5	13
3	6	14
4	1	-4
5	5	3
6	3	8
7	1	-9
8	4	2



4.3.4 用正交链表来表示稀疏矩阵



- 优点：适应矩阵操作(+、-、*)时矩阵非零元素的动态变化
- 稀疏矩阵表示为行链表与列链表的十字交叉——称为正交链表（十字链表）



• 稀疏矩阵的结点

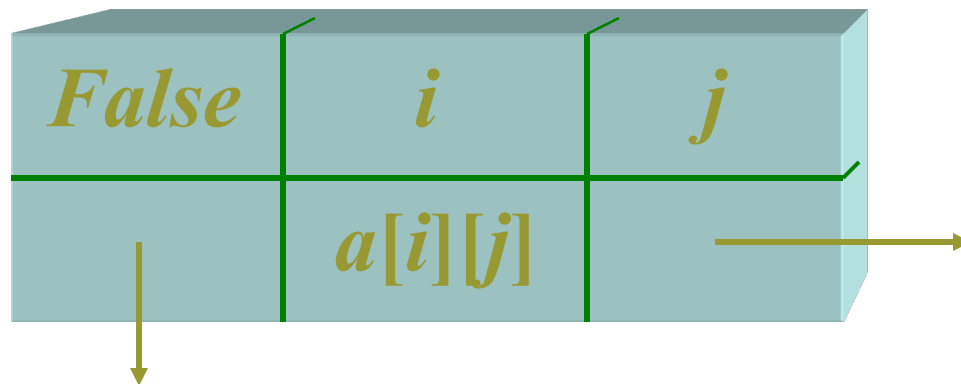


<i>head</i>	<i>next</i>
<i>down</i>	<i>right</i>

(a) 表头结点

<i>head</i>	<i>row</i>	<i>col</i>
<i>down</i>	<i>value</i>	<i>right</i>

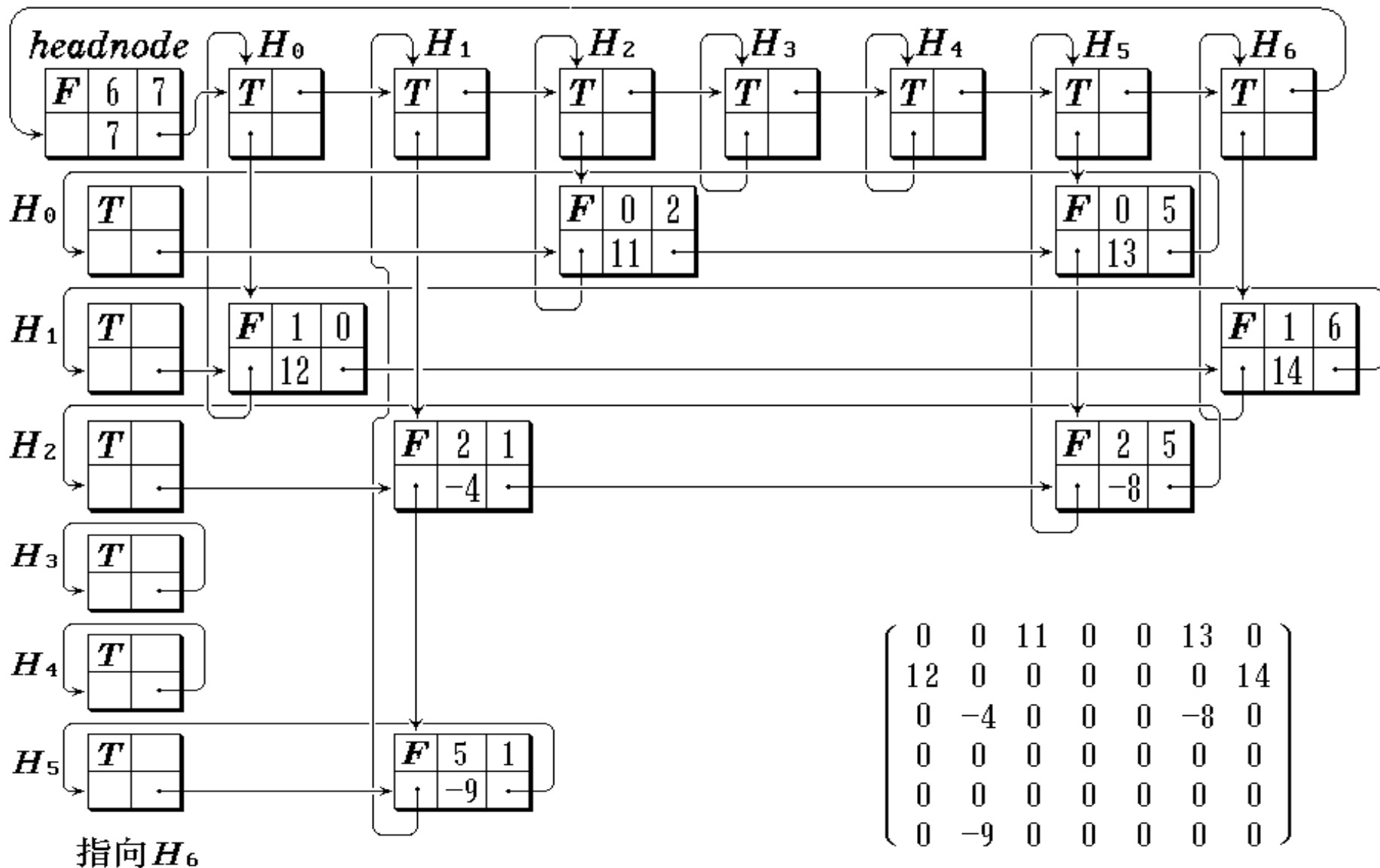
(b) 非零元素结点



(c) 建立 $a[i][j]$ 结点



稀疏矩阵的正交链表表示的示例

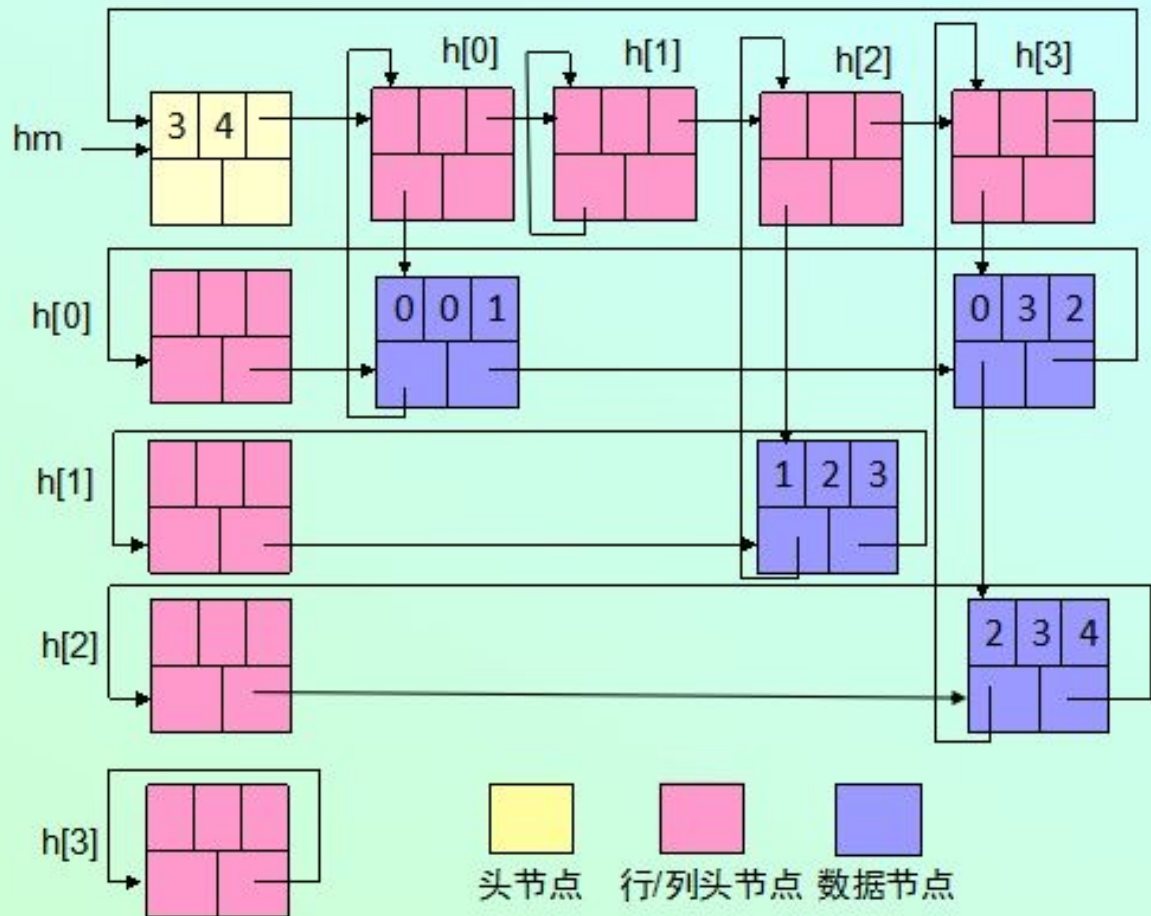




十字链表

- 只保存非零值
- 为每一行设置一个单独链表，同时也为每一列设置一个单独链表。

$$B_{3 \times 4} = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix}$$





特殊矩阵 压缩存储

对称矩阵

特点：对方阵中的任意一个元素，有 $a_{i,j} = a_{j,i}$

压缩：只存储主对角线+下三角区（或主对角线+上三角区）

三角矩阵

特点：上三角区全为常量（下三角矩阵）；或下三角区全为常量（上三角矩阵）

压缩：按行优先/列优先规则依次存储非常量区域，并在最后一个位置存放常量c

三对角矩阵
(带状矩阵)

特点：当 $|i - j| > 1$ 时，有 $a_{i,j} = 0$ ($1 \leq i, j \leq n$)

压缩：按行优先/列优先规则依次存储带状区域

稀疏矩阵

非零元素个数远小于零元素个数

压缩：只存储非零元素

顺序存储：顺序存储三元组 <行,列,值>

链式存储：十字链表法